

Ontolith

Setup & Installation Guide

Enterprise Ontology Control Plane

Prepared for local development and manual GitHub upload

Manus AI · August 2026

Contents

1 Overview	1
2 Prerequisites	1
3 Project structure	1
4 Installation	1
4.1 1. Obtain the project	1
4.2 2. Install dependencies	2
4.3 3. Start the development server	2
5 Using the test workspace	2
6 Validation and tests	2
7 Configuration and security	3
8 Manual GitHub upload	3
9 Production roadmap	3
10 Troubleshooting	3
10.1 pnpm is not found	3
10.2 The preview does not load	3
10.3 GitHub push returns HTTP 403	3
10.4 TypeScript or test failures appear after edits	4
11 License and contribution note	4

1 Overview

Ontolith is a browser-based enterprise data ontology and knowledge graph workspace. The preview application demonstrates multi-source schema ingestion, entity and relationship inference, interactive graph inspection, schema drift detection, conflict resolution, graph traversal queries, and ingestion history. The current implementation keeps preview snapshots and run history in memory. It is designed as a testable foundation for future PySpark ingestion workers, FastAPI boundaries, Neo4j persistence, durable schema versioning, and governance workflows.

2 Prerequisites

Install the following before starting local development:

- Node.js 22 or a compatible current LTS release.
- pnpm 10 or a compatible pnpm release.
- Git for source control.
- A modern Chromium, Firefox, or Safari browser.

The project does not require Neo4j, PySpark, Docker, or a production database to run the current browser preview. Those integrations are planned extension points rather than local prerequisites for the included test workspace.

3 Project structure

The main application files are organized as follows:

client/src/pages/Home.tsx	Dashboard and interactive workspaces
client/src/lib/ontology.ts	Parsing, inference, drift, conflicts, traversal
client/src/lib/ontology.test.ts	Core ontology unit tests
client/src/index.css	Blueprint visual system
server/	Express, tRPC, auth, and database scaffolding
drizzle/	Database schema and migration workspace
shared/	Shared constants and types
README.md	Product and developer overview
todo.md	Implementation checklist

4 Installation

4.1 1. Obtain the project

Extract the project archive into a working directory, or clone the GitHub repository after it has been uploaded manually:

```
git clone https://github.com/pavanvzm/Enterprise-Data-Ontology-Knowledge-Graph.git
cd Enterprise-Data-Ontology-Knowledge-Graph
```

If you are working from the ZIP archive, enter the extracted project directory instead.

4.2 2. Install dependencies

Install the locked dependency tree with pnpm:

```
pnpm install
```

Do not commit local `.env` files, access tokens, session secrets, or other credentials. Use the environment configuration provided by the deployment environment or create a local ignored environment file when required by your setup.

4.3 3. Start the development server

Run the full-stack development server:

```
pnpm dev
```

Open the preview URL printed by the development server. The application starts at the managed port assigned by the runtime; do not hardcode a port in deployment configuration.

5 Using the test workspace

The left navigation exposes the primary workflows:

- **Overview** presents mapped entities, relationships, drift alerts, recent ingestion activity, and the canonical graph.
- **Ingestion** accepts JSON and SQL schema text plus CSV, JSON, or SQL file selection. Run ingestion to create an in-memory snapshot.
- **Ontology graph** supports node selection, entity inspection, zoom controls, and pointer-drag panning.
- **Schema drift** compares versioned snapshots and displays added, removed, and changed fields or relationships.
- **Conflict resolver** presents conflicting field definitions, source provenance, confidence, and a merged schema.
- **Query workspace** runs simple graph traversal queries such as find all Orders linked to a Customer.
- **Ingestion history** lists source names, timestamps, versions, entity counts, and drift alerts.

6 Validation and tests

Run the TypeScript checker and Vitest suite before making changes or uploading the project:

```
pnpm check  
pnpm test
```

The tests cover JSON and CSV parsing, schema drift detection, ontology inference, conflict merging, graph traversal, and the scaffolded authentication logout procedure.

Run a production build to validate the deployable bundle:

```
pnpm build
```

7 Configuration and security

The scaffolded runtime supports authentication, database access, storage, analytics, and built-in services through environment variables. Never hardcode credentials in source files. Do not paste GitHub tokens, API keys, or secret values into chat, commit history, README files, or shell command arguments.

For GitHub uploads, use a fine-grained personal access token limited to the intended repository with **Contents: Read and write** and **Metadata: Read-only**, or use an approved GitHub connector. If a token is exposed, revoke it immediately and create a replacement through a secure flow.

8 Manual GitHub upload

If automated GitHub push is blocked by a 403 permission error, upload the project manually:

1. Create or open pavanvzm/Enterprise-Data-Ontology-Knowledge-Graph on GitHub.
2. Extract the full project archive locally.
3. Upload the project files while preserving the client, server, drizzle, shared, and configuration directories.
4. Preserve the included README.md, LICENSE, package.json, pnpm-lock.yaml, and test files.
5. Commit the upload to the repository's main branch.
6. Confirm that no .env files or tokens are included.

9 Production roadmap

For production use, add durable tables for schema snapshots and ingestion runs, a FastAPI or tRPC ingestion boundary, PySpark jobs for large source systems, Neo4j graph persistence, scheduled drift monitoring, role-based governance, approval workflows, and downloadable schema contracts. Preserve the vocabulary already used by the preview: entities, fields, relations, source datasets, snapshots, drift items, conflicts, and ingestion runs.

10 Troubleshooting

10.1 pnpm is not found

Install pnpm through the official Node.js package manager workflow, then reopen the shell and verify with `pnpm --version`.

10.2 The preview does not load

Check the development server output for startup errors, confirm that dependencies were installed, and restart with `pnpm dev`. Do not launch a second server on the same managed port.

10.3 GitHub push returns HTTP 403

Verify that the active credential belongs to the intended account, explicitly includes the target repository, and has **Contents: Read and write**. Refresh the secure connector or use an interactive GitHub CLI login. Never solve the issue by pasting a token into chat or embedding it in a remote URL.

10.4 TypeScript or test failures appear after edits

Run `pnpm check` first to identify type errors, then run `pnpm test`. Repair the first syntax or type error before diagnosing downstream failures.

11 License and contribution note

The repository includes its existing license file. Review the license terms before publishing a derivative or open-source distribution. Add contribution standards, issue templates, and a security policy before accepting external contributions.